

Comparison of Driver Performance

Driver 1 demonstrates better overall performance than Driver 2, with higher acceleration, greater peak speeds and higher speeds through corners.

Before Corner 1, Driver 1 reaches a top speed of 67 km/h, compared with 60 km/h for Driver 2, while also operating at a higher engine RPM. This suggests that Driver 1 accelerates more effectively on the approach to the corner.

The lap-time data further supports this conclusion. Driver 1 records the fastest interval of 64.904 seconds, while Driver 2's fastest interval is 72.471 seconds, a difference of 7.567 seconds. This is a significant gap, indicating that Driver 1 is able to maintain a higher overall pace.

The GPS speed data also highlights the difference in performance. Driver 1 has a mean GPS speed of 41.20 km/h and a maximum speed of 78.34 km/h, compared with Driver 2's mean of 37.43 km/h and maximum of 76.01 km/h. Driver 1 also has a higher standard deviation of speed (15.13 compared with 13.82), indicating greater variation in speed throughout the session. This suggests more aggressive acceleration and deceleration.

Driver 1 reaches a maximum lateral acceleration of 1.505 g, compared with 1.331 g for Driver 2. This suggests that Driver 1 can generate greater lateral acceleration through corners, potentially allowing higher cornering speeds. Driver 1 also reaches a maximum longitudinal acceleration of 0.864 g, compared with 0.616 g for Driver 2, indicating stronger peak acceleration under acceleration. These differences suggest that Driver 1 makes greater use of the car's available grip and performance.

Driver 1's throttle data supports this observation. The mean throttle position is 34.78%, compared with 27.17% for Driver 2. This suggests that Driver 1 applies more throttle on average and may accelerate more aggressively out of corners.

Braking data also shows a difference between the drivers. Driver 1 has a mean brake pressure of 3.51 bar and a maximum of 42.3 bar, while Driver 2 has a mean of 2.75 bar and a maximum of 36.5 bar. This indicates that Driver 1 uses higher braking pressures overall and reaches a higher peak braking pressure. Combined with the higher speeds approaching corners, this may suggest that Driver 1 brakes harder and potentially later. However, to confirm later braking, the braking points and vehicle speeds must be compared at the same locations on the track.

One particularly interesting finding is that Driver 1 has a higher vehicle mass due to the fuel load. A higher mass would normally reduce acceleration for a given net driving force and increase the force required to achieve the same deceleration. Therefore, Driver 1's stronger acceleration and higher peak speeds are particularly notable despite this disadvantage. If Driver 2 had the higher mass, it could potentially help explain the lower acceleration and deceleration observed in the data. Since the opposite is true, vehicle mass alone is unlikely to explain the performance difference.

Overall, the data suggests that Driver 1 is the faster and more aggressive driver, with stronger acceleration, higher cornering speeds and greater peak braking pressure. Driver 1 also achieves a substantially faster lap interval despite carrying a higher fuel load. Although differences in vehicle setup, track conditions and driving style must be considered.

Use Of AI

Prompt: optimise this code

```
df = {'driverOneDf': pd.read_csv(files[0]), 'driverTwoDf': pd.read_csv(files[1])}

print(df['driverOneDf'].shape)

print(df['driverTwoDf'].shape)

#Splitting into Laptimes

cumLapTimes = {

    'driverOneCumulativeTime': [113.901, 190.353, 261.857, 331.104, 398.652,

                                466.026, 536.357, 605.743, 671.024, 736.401,

                                808.066, 873.338, 938.436, 1003.34, 1083.0],

    'driverTwoCumulativeTime': [40.594, 117.581, 191.908, 264.67, 338.821,

                                413.768, 487.82, 561.371, 636.068, 708.539,

                                783.379, 862.935, 940.344, 1036.0]

}

lapTimes={

    "driverOne": [113.901, 76.452, 71.504, 69.247, 67.548, 67.374, 70.331,

                  69.386, 65.281, 65.377, 71.665, 65.272, 65.098, 64.904, 79.66],

    "driverTwo": [40.594, 76.987, 74.327, 72.762, 74.151, 74.947, 74.052,

                  73.551, 74.697, 72.471, 74.84, 79.556, 77.409, 95.656],

}
```

```

#Scatter plot if each colluimn against time, by lap
for header in df['driverOneDf'].columns:
    for d, cumTime in zip(df, cumLapTimes):
        df[d].loc[:, 'Time (s)'] < cumTime, = df[d]['Time (s)'] - cumTime
    ax = df['driverOneDf'].plot.scatter(x="Time (s)", y=header, color="DarkBlue")
    df['driverTwoDf'].plot.scatter(x="Time (s)", y=header, color="DarkRed", ax = ax)
plt.show()

```

Response:

Here's the cleaned-up version. The main issues: the hardcoded lapTimes can just be derived from cumLapTimes with `np.diff`, the zip over two dicts was pairing keys and then comparing lengths mismatched (15 vs 14 entries), and the line `df[d].loc[:, 'Time (s)'] < cumTime, = ...` isn't valid Python — looks like you wanted lap-based filtering, but you never actually use it.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def compare_drivers(files):
    drivers = {
        'driverOne': pd.read_csv(files[0]),
        'driverTwo': pd.read_csv(files[1]),
    }
    for name, d in drivers.items():
        print(f"{name}: {d.shape}")
    # Lap boundaries – first lap ends at the first entry, etc.
    cumLapTimes = {
        'driverOne': [113.901, 190.353, 261.857, 331.104, 398.652,
                      466.026, 536.357, 605.743, 671.024, 736.401,
                      808.066, 873.338, 938.436, 1003.34, 1083.0],
        'driverTwo': [40.594, 117.581, 191.908, 264.67, 338.821,
                      413.768, 487.82, 561.371, 636.068, 708.539,
                      783.379, 862.935, 940.344, 1036.0],
    }
    # Derive individual lap times instead of hardcoding them
    lapTimes = {
        name: np.diff([0.0, *times]).round(3).tolist()
    }
    for name, times in cumLapTimes.items():
        print(lapTimes)
    # -> driverOne: [113.901, 76.452, 71.504, ...] (matches your hardcoded list)
    # Assign each telemetry row to a lap number, then plot per column
    for name, d in drivers.items():
        d['Lap'] = np.searchsorted(cumLapTimes[name], d['Time (s)'], side='right') + 1
    columns = [c for c in drivers['driverOne'].columns if c not in ('Time (s)', 'Lap')]
    for header in columns:
        fig, ax = plt.subplots(figsize=(10, 6))
        for name, color in zip(drivers, ('DarkBlue', 'DarkRed')):
            d = drivers[name]
            for lap, grp in d.groupby('Lap'):
                ax.scatter(grp['Time (s)'], grp[header], color=color, label=f"{name} - lap {lap}" if lap == 1 else None)
            ax.set(xlabel='Time (s)',

```

```

ylabel=header, title=f"{header} vs Time")          ax.legend()
plt.show() compare_drivers(files)

```

Prompt:

adjust to include heat map of 3rd column or i in range (len(columns)):

```

print(i, ':', columns[i])
header = columns[int(input("Enter Header index: "))]
heat = columns[int(input("Enter index for heat mapping: "))]
# Plot
fig, ax = plt.subplots(figsize=(10, 6))
for name, color in zip(drivers, ('DarkBlue', 'DarkRed')):
    d = drivers[name]
    smoothedY = gaussian_filter1d(d[header], sigma=15) #smooth the y axis by using gaussian
    ax.plot(d['Time (s)'], smoothedY, color=color, label=f"{name}")
    ax.plot(d['Time (s)'], d[header], color=color, label=f"{name}", alpha = 0.3)

    #Plot heat map, with normalised data

    norm_y = normalize(d[header], axis=1, norm='l1')

    # for lap, grp in d.groupby('Lap'):

    #     ax.plot(grp['Time (s)'], grp[header], color=color,

    #             label=f"{name} - lap {lap}" if lap == 1 else None)

ax.set(xlabel='Time (s)', ylabel=header, title=f"{header} vs Time")

ax.legend()

plt.show()

```

Response:

```

from scipy.ndimage import gaussian_filter1d from matplotlib.collections
import LineCollection from matplotlib.colors import LinearSegmentedColormap
# Driver 1: green -> red-orange cmap1 =
LinearSegmentedColormap.from_list('d1', ['#00c000', '#ff8c00'])# Driver 2:
blue-green -> red cmap2 = LinearSegmentedColormap.from_list('d2', ['#00b8b8',
'#ff3030']) fig, ax = plt.subplots(figsize=(12, 6)) lcs = []for name, cmap
in zip(drivers, (cmap1, cmap2)):    d = drivers[name]    x = d['Time (s)']
y = gaussian_filter1d(d[header], sigma=15)    h = d[heat].values    norm_h
= (h - h.min()) / (np.ptp(h) or 1)    # 0% -> 100% per driver    pts =
np.column_stack([x, y]).reshape(-1, 1, 2)    segments =

```

```

np.concatenate([pts[:-1], pts[1:]], axis=1)      lc = LineCollection(segments,
cmap=cmap, linewidth=3,                        array=norm_h[:-1],
label=name)      ax.add_collection(lc)      lcs.append((lc, heat, h.min(),
h.max()))  ax.set(xlabel='Time (s)', ylabel=header, title=f"{header} vs Time
(colour = {heat})") ax.set_xlim(x.min(), x.max()) ax.set_ylim(y.min(),
y.max()) ax.legend()# One colourbar per driver, labelled with real values
cbar1 = fig.colorbar(lcs[0][0], ax=ax, pad=0.02) cbar1.set_ticks([0, 1]);
cbar1.set_ticklabels([f"{lcs[0][2]:.1f}", f"{lcs[0][3]:.1f}"])
cbar1.set_label(f"{name} {heat}") cbar2 = fig.colorbar(lcs[1][0], ax=ax,
pad=0.06) cbar2.set_ticks([0, 1]); cbar2.set_ticklabels([f"{lcs[1][2]:.1f}",
f"{lcs[1][3]:.1f}"]) cbar2.set_label(f"{name} {heat}") plt.show()

```

Used Ai to help smooth english response.